

Introduction à la programmation en R

Probabilités et Statistique (info/phys)

Méthodes statistiques (math)

Souleymane THIANDOUM Matthieu SIMON



Nous tenons à remercier chaleureusement notre collègue, le Professeur **Karl GROSSE-ERDMANN**, pour avoir généreusement partagé ces supports de cours.

1 Introduction

C'est quoi R ?

- R est un langage de programmation pour le traitement de données et la préparation de graphiques.
- R est basé sur le langage de programmation S (créé en 1976). S-PLUS est une implémentation commerciale de S, et R est sa contrepartie « Open Source ».
- R est un langage orienté objet, interprété (et non compilé) et basé sur la notion de vecteur. Ses fonctionnalités peuvent être augmentées grâce à de nombreux packages.

Un peu de culture

R a été créé en 1993 par **Ross Ihaka** et **Robert Gentleman** à l'Université d'Auckland en Nouvelle-Zélande, et est maintenant développé par le R Development Core Team.

L'origine du nom du langage provient, d'une part, des initiales des prénoms des deux auteurs (Ross et Robert) et, d'autre part, d'un jeu de mots sur le nom du langage S auquel il est apparenté.

Littérature :

P.-A. Cornillon et autres : Statistique avec R, 3ème édition, PU Rennes 2012.

F. Bertrand, M. Maumy-Bertrand : Initiation à la statistique avec R, 3ème édition, Dunod 2018.

Installation

Télécharger et installer la distribution R via CRAN
(Comprehensive R Archive Network) :

<http://cran.r-project.org/>

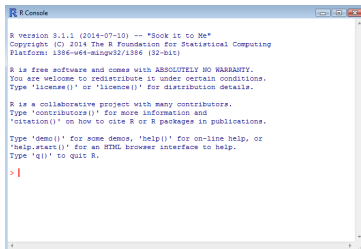
Cette distribution vient avec une console pour travailler avec R, mais qui n'est pas très maniable. Il est donc recommandé d'utiliser un éditeur pour R :

Télécharger et installer l'éditeur RStudio :

<https://www.rstudio.com/products/rstudio/download/>

Utiliser R dans une console

Si on ouvre une session de travail sous R, la console apparaît.



```
R Console

R version 3.1.1 (2014-07-10) -- "Suck it to Me"
Copyright (C) 2014 The R Foundation for Statistical Computing
Platform: i386-w64-mingw32/x86_64 (32-bit)

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

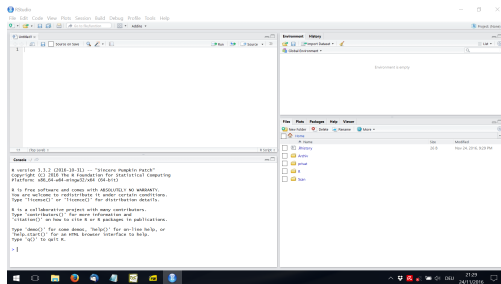
Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

> |
```

Mais l'utilisation directe de la console n'est pas conseillée car elle est assez inflexible.

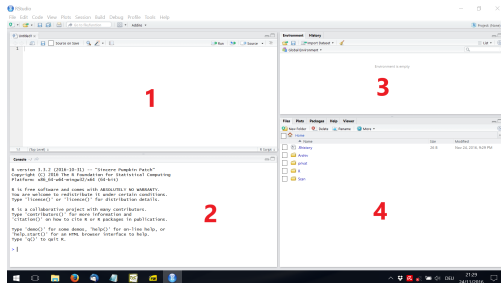
Utiliser R dans RStudio (1/4)

Si on ouvre une session de RStudio on voit quatre fenêtres.



Attention : Lors du premier appel de RStudio, on voit seulement trois fenêtres. Il faut réduire la fenêtre à gauche pour faire apparaître la quatrième fenêtre.

Utiliser R dans RStudio (2/4)



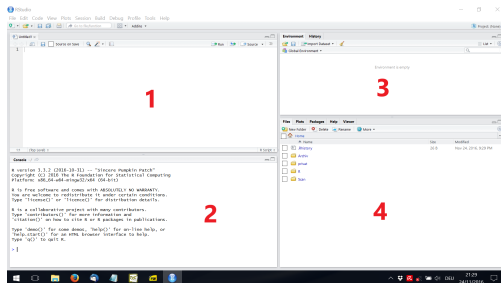
Dans la fenêtre **1** on écrit les commandes.

Pour exécuter la ligne où se trouve le curseur, il suffit de cliquer sur l'icône « Run » (flèche dirigée vers la droite). Pour exécuter plusieurs lignes à la fois il faut les sélectionner.

On peut sauvegarder le code dans un fichier avec l'extension **.R**.



Utiliser R dans RStudio (3/4)



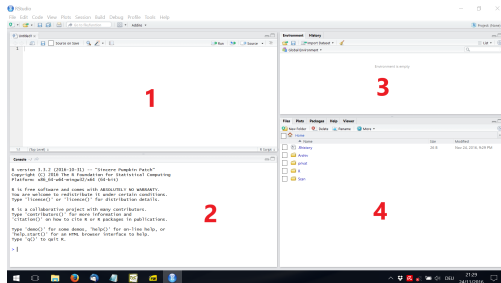
La fenêtre **2** contient la console de R. On y voit les résultats du travail.

Le symbol « > » est l'invite de commande de R. Si par contre le symbole « + » apparaît, la commande n'est pas encore complète. Cela indique généralement une faute dans le code.

Attention : Dans le code de la fenêtre **1** on n'écrit pas le symbole « > ».



Utiliser R dans RStudio (4/4)



La fenêtre **3** contient l'environnement. On y voit tous les objets créés avec leurs paramètres et leurs valeurs.

On peut sauvegarder les objets avec leurs valeurs dans un fichier avec l'extension `.RData`.

La fenêtre **4** contient, entre autres, les plots et un accès à l'aide.



Quelques fonctionnalités de R

- *R comme calculatrice*

Tapez la ligne de commande suivante :

```
> 2 + 3
```

Le résultat s'affiche sous la forme :

```
[1] 5
```

- *R comme éditeur graphique*



- *R comme logiciel de statistique*
- *N'oubliez pas les packages !*



Obtenir de l'aide

Dans la fenêtre **4** de RStudio, vous avez accès à une aide sur toutes les fonctions de R : Sous l'onglet **Help** vous pouvez entrer la fonction que vous cherchez.

Par exemple, la fonction **matrix** permet de créer des matrices. L'aide vous explique l'usage de la fonction, les arguments, et vous donne des exemples.

On peut également utiliser la commande

```
> help(matrix)
```

② Bases du langage R

Affectations et expressions (1/2)

Toute commande dans R est soit une expression, soit une affectation.

Une **expression** est immédiatement évaluée et le résultat est affiché à l'écran :

```
> pi  
[1] 3.141593  
> sin(pi)  
[1] 1.224606e-16
```

Par défaut, les résultats sur R sont donnés avec 6 à 7 décimales et la précision des calculs est assez limitée.

Affectations et expressions (2/2)

Lors d'une **affectation**, une expression est évaluée, mais le résultat est stocké dans un objet (variable) et rien n'est affiché à l'écran.

Le symbole d'affectation est `<-` :

```
> a <- 7
> a
[1] 7
> b <- a
> b
[1] 7
```

L'affectation inverse `->` existe aussi, mais elle est rarement utilisée.

Évitez d'utiliser `=` (même si ça fonctionne également).

Pour affecter le résultat d'un calcul **et** voir le résultat, on peut placer l'affectation entre parenthèses.

```
> (a <- 2 + 7)
[1] 9
```



Objets (1/2)

Les objets dans R se caractérisent par :

- un nom,
- une classe (vecteur, matrice, array, data-frame, liste, ...),
- un mode (numeric, complex, character, logical, NULL, ...),
- une longueur (nombre d'éléments contenus dans l'objet).

```
> v <- c(1, 2, 5, 9)
> mode(v)
[1] "numeric"
> length(v)
[1] 4
> w <- c("a", "bc")
> length(w)
[1] 2
> mode(w)
[1] "character"
```



Objets (2/2)

On voit tous les objets créés lors d'une session dans la fenêtre **3** de RStudio. La liste inclut leurs paramètres (nom, longueur, valeurs, ...)

Dans cette fenêtre on peut également supprimer quelques objets ou même tous les objets.

Rappel : On peut sauvegarder les objets avec leurs valeurs dans un fichier avec l'extension `.RData`.

Noms d'objets (1/3)

Les caractères permis dans les noms d'objets sont :

- les lettres `a`, `b`, ..., `z`, `A`, `B`, ..., `Z`,
- les chiffres `0`, `1`, ..., `9`,
- le point « `.` »,
- le « `_` ».

Les noms d'objets ne peuvent pas commencer par un chiffre ou l'underscore.

R est sensible à la casse, ce qui signifie que

`nom`, `Nom`, `NOM`

sont trois objets distincts.

Exemples de noms :

`a`, `b`, `ex1`, `ex2.1`, `Proximus_1_1_2008`, `STAT_1`, ...

Noms d'objets (2/3)

Certains noms sont utilisés par le système, aussi vaut-il mieux éviter de les utiliser :

`c`, `q`, `t`, `C`, `D`, `I`, `T`, `F`, `pi`, ...

Certains mots sont réservés pour le système et il est interdit de les utiliser comme nom d'objet :

`TRUE`, `FALSE`, `NA`, `NaN`, `NULL`, `if`, `next`, ...

Noms d'objets (3/3)

Par exemple, les noms `TRUE` et `FALSE` sont réservés (ce sont des éléments logiques).

Les variables `T` et `F` prennent par défaut les valeurs `TRUE` et `FALSE`, respectivement, mais peuvent être réaffectées.

```
> T
[1] TRUE
> T <- 3
> T
[1] 3
> TRUE <- 3
Error in TRUE <- 3 : invalid (do_set) left-hand side to
assignment
```



Mode de vecteurs, matrices et tableaux

Toutes les entrées dans un même vecteur (une matrice, un tableau) doivent être du même mode. Si on mélange des modes différents, les données seront converties en un même mode (selon une certaine règle).

Pour convertir le mode d'un objet en un autre mode, utilisez

`as.numeric()`, `as.complex()`, ...

Pour tester le mode d'un objet, utilisez

`is.numeric()`, `is.complex()`, ...

```
> a <- c(1, 2, "3")
> a
[1] "1" "2" "3"
> is.character(a)
[1] TRUE
> (anum <- as.numeric(a))
```



Objets spéciaux : NA et NULL

NA est fréquemment utilisé pour représenter les données manquantes.

- son mode est `logical`,
- sa longueur est 1,
- une opération impliquant une donnée NA rend, en général, comme résultat NA,
- avec l'option `na.rm = TRUE`, on peut éliminer les données manquantes avant de faire un calcul,
- la fonction `is.na()` teste si un objet contient des éléments NA.

NULL représente "rien" ou le vide.

- son mode est `NULL`,
- sa longueur est 0,
- la fonction `is.null()` teste si un objet est NULL.



Commentaires

Le symbole `#` permet d'ajouter des commentaires dans le code.
Ce qui suit le symbole `#` n'est pas interprété par R.

```
> a<-NULL # crée l'objet a de valeur NULL
```

Vecteurs (1/5)

En R, l'objet de base est le vecteur.

Un scalaire est un vecteur de longueur 1.

Vecteurs (2/5)

Pour créer un vecteur, différentes méthodes sous R sont disponibles dont voici quelques-unes :

- la fonction `c` (c comme concaténation)

```
> (ex1 <- c(1,3,5,7))  
[1] 1 3 5 7
```

- l'opérateur :

```
> (ex2 <- 10:40)  
[1] 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25  
[17] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40
```

- la fonction `seq` (seq comme séquence) avec comme paramètres le départ, l'arrivée et optionnellement le pas

```
> (ex3 <- seq(1,9,2))  
[1] 1 3 5 7 9
```

- la fonction `rep`

```
> (ex4 <- rep(c(1,2),3))  
[1] 1 2 1 2 1 2
```



Vecteurs (3/5)

Il est possible (et souvent souhaitable) de donner une étiquette à chacun des éléments d'un vecteur.

```
> (ex1 <- c(a=1,b=3,c=5,d=7))  
a b c d  
1 3 5 7
```

Avec la commande `names()` on peut ajouter des étiquettes à un vecteur :

```
> (ex3 <- seq(1,9,2))  
[1] 1 3 5 7 9  
> names(ex3) <- c("a", "b", "c", "d", "e")  
> ex3  
a b c d e  
1 3 5 7 9
```

Avec la commande `names() <- NULL` on peut supprimer



Vecteurs (4/5)

L'extraction d'éléments dans un vecteur se fait avec `[ ]`. On peut

- extraire des éléments d'un vecteur à l'aide des positions

```
> ex3[2:3]  
b c  
3 5
```

- extraire des éléments d'un vecteur à l'aide de ses étiquettes (si elles existent)

```
> ex3["c"]  
c  
5
```

- supprimer des éléments avec la commande `-`

```
> ex3[-(2:3)]  
a d e  
1 7 9
```



Vecteurs (5/5)

- extraire les éléments en inversant l'ordre

```
> ex3[5:1]  
e d c b a  
9 7 5 3 1
```

- extraire les éléments qui satisfont une condition spécifiée

```
> ex3[ex3<8]  
a b c d  
1 3 5 7
```

- extraire les premiers éléments d'un vecteur avec la fonction `head`

```
> head(ex3, 3)  
a b c  
1 3 5
```

- extraire les derniers éléments d'un vecteur avec la fonction `tail`

```
> tail(ex3, 3)  
c d e  
5 7 9
```



Matrices (1/5)

Un **tableau** est un vecteur doté d'un attribut `dim`. Il est stocké sous forme de vecteur. En particulier, une **matrice** est un tableau de dimension 2.

Les fonctions de base pour créer des matrices et tableaux sont respectivement `matrix` et `array`.

On explique ici seulement les matrices. Les tableaux se traitent de façon similaire.

```
> (m<-matrix(1:12,ncol=3))  
      [,1] [,2] [,3]  
[1,]    1    5    9  
[2,]    2    6   10  
[3,]    3    7   11  
[4,]    4    8   12
```

On remarque que les éléments de la matrice sont rangés par colonne

Matrices (2/5)

Les valeurs données à `ncol` ou `nrow` doivent être des multiples ou des diviseurs du nombre de données. Si c'est un multiple, les données seront répétées. Si ces consignes ne sont pas respectées, R crée une matrice selon une certaine règle.

Avec les fonctions `rownames` et `colnames` on peut attribuer des étiquettes à une matrice.

```
> a<-matrix(1:6,nrow=2),  
> rownames(a)<-c("Gauche","Droite")  
> colnames(a)<-c("A","B","C")  
> a
```

	A	B	C
Gauche	1	3	5
Droite	2	4	6

On aura le même résultat avec la fonction `dimnames`

```
> dimnames(a) <- list(c("Gauche","Droite"),c("A","B","C"))
```



Matrices (3/5)

Obtenir des éléments d'une matrice se fait également avec `[ ]`.

On extrait les éléments en précisant leurs positions dans la matrice, ou encore leurs positions dans le vecteur sous-jacent.

```
> (m <- matrix(c(40, 80, 45, 21, 55, 32), 2, 3))  
      [,1] [,2] [,3]  
[1,]   40   45   55  
[2,]   80   21   32  
> m[1,2]  
[1] 45  
> m[3]  
[1] 45
```

Pour extraire des lignes ou des colonnes :

```
> m[,3]  
[1] 55 32
```

Matrices (4/5)

On voit que le résultat de l'extraction est un vecteur.

```
> (m <- matrix(c(40, 80, 45, 21, 55, 32), 2, 3))
      [,1] [,2] [,3]
[1,]   40   45   55
[2,]   80   21   32
> m[,3]
[1] 55 32
```

Si on veut obtenir une matrice il faut utiliser l'option `drop=F`.

```
> m[,3,drop=F]
      [,1]
[1,]   55
[2,]   32
```

```
> m[2, ,drop=F]
      [,1] [,2] [,3]
[1,]   80   21   32
```



Matrices (5/5)

On peut également extraire selon une condition.

```
> (m <- matrix(1:8, ncol=4, byrow=T))  
      [,1] [,2] [,3] [,4]  
[1,]    1    2    3    4  
[2,]    5    6    7    8
```

On extrait les colonnes dont le premier élément est strictement supérieur à 2.

```
> m[,m[1,]>2]  
      [,1] [,2]  
[1,]    3    4  
[2,]    7    8
```

Facteurs

Un **facteur** est un vecteur particulier qui permet de manipuler des variables qualitatives.

Les facteurs peuvent être non ordonnés (ex : mâle, femelle) ou ordonnés (ex : ceintures de judo).

Pour créer un facteur, on utilise la fonction `factor`.

```
> (sexe <- factor(c("M", "M", "F", "M", "F", "M", "M", "M")))
[1] M M F M F M M M
Levels: F M
```

Avec `ordered` on ordonne les niveaux.

```
> judo <- factor(c("jaune", "verte", "noire", "verte"))
> ordered(judo, levels=c("jaune", "verte", "noire"))
[1] jaune verte noire verte
Levels: jaune < verte < noire
```



③ Opérateurs et fonctions

Généralités sur les fonctions (1/3)

Une fonction peut avoir plusieurs arguments qui sont séparés par une virgule.

```
> (a<-min(3,9))  
[1] 3
```

L'aide dans RStudio (sous l'onglet [Help](#)) vous donne entre autres les arguments d'une fonction.

Les arguments d'une fonction peuvent prendre deux formes : obligatoires ou optionnelles.

Les arguments optionnels possèdent une valeur par défaut qui est donnée après « = ».

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE,  
dimnames = NULL)
```

Généralités sur les fonctions (2/3)

```
matrix(data = NA, nrow = 1, ncol = 1, byrow = FALSE,  
dimnames = NULL)
```

Les valeurs par défaut sont utilisées si l'argument n'est pas spécifié dans l'appel de la fonction.

Ainsi, un appel à `matrix` sans argument résulte en une matrice 1×1 remplie de la valeur NA et dont les dimensions sont dépourvues d'étiquettes.

```
> matrix()  
      [,1]  
[1,]    NA
```

Généralités sur les fonctions (3/3)

L'ordre des arguments est important ; il est donc nécessaire de les nommer s'ils ne sont pas appelés dans l'ordre.

```
> matrix(1:6,2,3,T)
      [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
```

Le même résultat sera obtenu avec la commande

```
> matrix(1:6,byrow=T,ncol=3,nrow=2)
```

Opérations sur les vecteurs

Les opérations sur les vecteurs sont effectuées **élément par élément**.

```
> c(1, 2, 3) + c(4, 5, 6)
[1] 5 7 9
> 1:3 * 4:6
[1] 4 10 18
```

Si les vecteurs ne sont pas de la même longueur, les plus courts sont recyclés de façon à correspondre au plus long vecteur.

```
> 1:10 + 2
[1] 3 4 5 6 7 8 9 10 11 12
> 1:10 + 2:3
[1] 3 5 5 7 7 9 9 11 11 13
```

Principaux opérateurs mathématiques

En ordre décroissant de priorité des opérations, on a :

$\wedge$ ou $**$	puissance
$-$	changement de signe
$*$, $/$	multiplication, division
$+$, $-$	addition, soustraction
$\% \%$, $\% / \%$	modulo, division entière
$\% * \%$	produit de matrices
$<$, $<=$, $=$, $>=$, $>$, $!=$	plus petit, plus petit ou égal, égal, plus grand ou égal, plus grand, différent de
$!$	négation logique
$\&$, $ $	« et » logique, « ou » logique

Gestion de données

<code>rep</code>	créer un vecteur en répétant des éléments
<code>scan</code>	créer un vecteur au fur et à mesure
<code>sort</code>	trier par ordre croissant (ou décroissant)
<code>rev</code>	renverser un vecteur
<code>which</code>	positions des valeurs qui satisfont une condition donnée
<code>which.min</code>	position du minimum
<code>which.max</code>	position du maximum
<code>match</code>	position de la première occurrence d'un élément
<code>%in%</code>	appartenance d'une ou plusieurs valeurs à un vecteur

Fonctions numériques

Les fonctions `sqrt`, `exp`, `sin`, `cos`, `tan`, `log`, `log10`, ... ont un sens évident.

<code>abs</code>	valeur absolue
<code>round</code>	arrondi à un nombre défini de décimales
<code>floor</code>	plus grand entier inférieur ou égal
<code>ceiling</code>	plus petit entier supérieur ou égal
<code>trunc</code>	troncature vers zéro ; différent de <code>floor</code> pour les nombres négatifs

Opérations sur les matrices

<code>rbind</code> , <code>cbind</code>	fusionner par ligne (colonne) de matrices
<code>t</code>	transposée
<code>det</code>	déterminant
<code>solve</code>	inverse ; résolution de systèmes d'équations linéaires
<code>eigen</code>	valeurs propres et vecteurs propres
<code>diag</code>	extraction de la diagonale d'une matrice ; création d'une matrice diagonale
<code>rowSums</code> , <code>colSums</code>	sommes par ligne, sommes par colonne
<code>rowMeans</code> , <code>colMeans</code>	moyennes par ligne, moyennes par colonne

Statistique descriptive

<code>sum, prod</code>	somme, produit
<code>diff</code>	différences entre nombres consécutifs
<code>mean</code>	moyenne arithmétique
<code>var, sd</code>	variance, écart type
<code>cv</code>	matrice de covariance
<code>cor</code>	matrice de corrélation
<code>min, max</code>	minimum, maximum
<code>range</code>	minimum et maximum
<code>median</code>	médiane
<code>quantile</code>	quantile
<code>summary</code>	statistiques descriptives usuelles

Sommaires cumulatifs et comparaisons élément par élément

<code>cumsum</code> , <code>cumprod</code>	somme cumulative, produit cumulatif
<code>cummin</code> , <code>cummax</code>	minimum et maximum cumulatif
<code>pmin</code> , <code>pmax</code>	minimum et maximum élément par élément (parallèle entre plusieurs vecteurs)

Data-frames (1/6)

Un **data-frame** est un objet qui est composé de plusieurs vecteurs. Ses composantes doivent avoir la **même longueur**, mais leurs modes peuvent être différents.

Les tableaux de données statistiques sont souvent encodées en data-frames.

La fonction de base pour créer des data-frames est `data.frame`.

```
> s <- c("M", "F", "F", "M")
> p1 <- c(19.5, 17.5, 12, 9)
> a2 <- c(18.5, 18, 8, 10)
> r <- c(TRUE, TRUE, FALSE, FALSE)
> notes <- data.frame(s, p1, a2, r)
> notes
```

	s	p1	a2	r
1	M	19.5	18.5	TRUE
2	F	17.5	18	TRUE
3	F	12	8	FALSE
4	M	9	10	FALSE



Data-frames (2/6)

Il est conseillé d'étiqueter les composantes d'un data-frame.

```
> names(notes) <- c("sexe", "Proba", "Ana", "réussite")
> notes
```

	sexe	Proba	Ana	réussite
1	M	19.5	18.5	TRUE
2	F	17.5	18.0	TRUE
3	F	12.0	8.0	FALSE
4	M	9.0	10.0	FALSE

On peut également étiqueter les individus.

```
> row.names(notes) <- c("Paul", "Aude", "Léna", "Marc")
> notes
```

	sexe	Proba	Ana	réussite
Paul	M	19.5	18.5	TRUE
Aude	F	17.5	18.0	TRUE
Léna	F	12.0	8.0	FALSE
Marc	M	9.0	10.0	FALSE



Data-frames (3/6)

Pour extraire une composante d'un data-frame, on peut toujours le faire en indiquant la position de l'élément que l'on souhaite extraire.

Les simples crochets `[ ]` retournent un data-frame :

```
> notes[2]
      Proba
Paul   19.5
Aude   17.5
Léna   12.0
Marc    9.0
```

Les doubles crochets `[[ ]]` retournent un vecteur (ou bien un facteur si le mode est caractère) :

```
> notes[[2]]
[1] 19.5 17.5 12.0 9.0
> notes[[1]]
[1] "M" "F" "F" "M"
Levels: F M
```



Data-frames (4/6)

On peut également extraire par l'étiquette...

```
> notes$Proba  
[1] 19.5 17.5 12.0 9.0
```

...ou un seul élément...

```
> notes[[2]][3]  
[1] 12  
> notes$Proba[4]  
[1] 9
```

...ou selon une condition (très utile !):

```
> subset(notes, Proba>17, select=c(sexe, Proba))  
      sexe  Proba  
Paul     M   19.5  
Aude     F   17.5
```



Data-frames (5/6)

Voici quelques opérations sur les data-frames :

<code>rbind</code>	fusionner par ligne de data-frames (le noms des colonnes doivent être identiques)
<code>cbind</code>	fusionner par colonne de data-frames (les noms des lignes sont ceux du premier data-frame)
<code>as.data.frame</code>	transformer une matrice en data-frame
<code>data.matrix</code>	transformer un data-frame en matrice

Data-frames (6/6)

Il est possible d'obtenir des informations sur chacune des variables d'un data-frame (ou une matrice) grâce à la fonction `summary`.

```
> summary(notes)
```

sexe	Proba	Ana	réussite
Length:4	Min. : 9.00	Min. : 8.00	Mode :logical
Class :character	1st Qu.:11.25	1st Qu.: 9.50	FALSE:2
Mode :character	Median :14.75	Median :14.00	TRUE :2
:	Mean :14.50	Mean :13.62	:
:	3rd Qu.:18.00	3rd Qu.:18.12	:
:	Max. :19.50	Max. :18.50	:

Listes

Une **liste** généralise les data-frames : les composantes d'une liste peuvent être de n'importe quel mode et **n'importe quelle longueur**.

La fonction de base pour créer des listes est `list`.

```
> n <- c(2,3,5)
> car <- c("aa", "bb", "cc", "dd", "ee", "aa", "bb")
> (maliste <- list(n, car))
[[1]]
[1] 2 3 5

[[2]]
[1] "aa" "bb" "cc" "dd" "ee" "aa" "bb"
```

L'extraction des éléments d'une liste se fait comme pour des data-frames.

Toutes les fonctions qui retournent plusieurs objets (par exemple, `eigen`) le font sous la forme d'une liste.

⑤ Importer et exporter des données

Importer et exporter des données (1/6)

Pour les lectures et écritures de fichiers, R utilise le répertoire de travail.

Pour connaître le répertoire de travail actuel, il suffit d'utiliser la commande `getwd()`.

```
> getwd()
[1] "C:/Users/MonNom/Documents"
```

Pour changer de répertoire de travail pour la durée de la session R, on peut utiliser la commande `setwd()`.

```
> setwd("C:/Users/MonNom/Documents/R")
```

Dans RStudio, on peut le changer également via le menu sous « Session ».

Importer et exporter des données (2/6)

En statistique, les données constituent le point de départ de toute analyse.

Elles sont souvent initialement collectées, voire prétraitées avec un logiciel de statistique ou un tableur (Excel, LibreOffice Calc, R, ...).

Chaque logiciel possédant son propre format de stockage, le plus simple est d'utiliser un format commun à tous, c'est-à-dire un format texte (`.txt` ou `.csv`).

Pour importer les données contenues dans le fichier, on utilise la commande `read.table`.

Importer et exporter des données (3/6)

Exemple : Pour importer un tableau (avec en-têtes et noms de ligne) :

```
> table <- read.table("fichier.csv", sep=";",  
+ header=T, dec=".", row.names=1)
```

- `sep` indique le caractère qui sépare les colonnes. Pour un espace, on utilise " " et pour une tabulation "\t".
- `header` indique si la première ligne contient ou non les intitulés des variables.
- `dec` indique le séparateur décimal.
- `row.names` indique la colonne qui contient les noms des lignes, ou bien spécifie les noms.

Si le jeu de données est enregistré ailleurs que dans le répertoire de travail R, il faut spécifier le chemin. Ce dernier peut aussi être une URL.

Importer et exporter des données (4/6)

Dans RStudio, il est facile d'importer un tableau de données : on clique sur le menu « Import Dataset » sous l'onglet « Environnement » de la fenêtre **3**, on se laisse guider.

Importer et exporter des données (5/6)

Si vous voulez **exporter** un tableau de données, vous utiliserez la commande suivante `write.table`.

```
> write.table(a, "fichier.csv", sep=";", col.names=F,  
+ row.names=F)
```

Un fichier `fichier.csv` contenant le tableau est alors créé dans le répertoire de travail de R.

Importer et exporter des données (6/6)

Rappel :

On peut sauvegarder les objets créés par R dans un fichier avec l'extension `.RData`. La prochaine fois que vous ouvrirez R vous pouvez alors charger ces objets avec leurs valeurs.

Dans RStudio, on utilise pour cela l'onglet « Environment » dans la fenêtre **3**.

Vous pouvez aussi utiliser les fonctions `save` et `load`, par exemple :

```
> save(a, file = "fichier.RData")
```

```
> load("fichier.RData")
```

⑥ Graphiques

Introduction

Avec R, vous pouvez produire des graphiques de toute nature. Pour un aperçu, tapez la commande suivante :

```
> demo(graphics)
```

Nous étudierons ici quelques graphiques de base.

La fonction `plot` (1/4)

La manière la plus simple de produire des graphiques sous R est d'utiliser la fonction `plot`.

L'utilisation classique de cette fonction consiste à représenter un nuage de points d'une variable y en fonction d'une variable x .

Par exemple, pour représenter à intervalles réguliers les valeurs de la fonction $x \mapsto \sin(2\pi x)$ sur $[0, 1]$, on écrira :

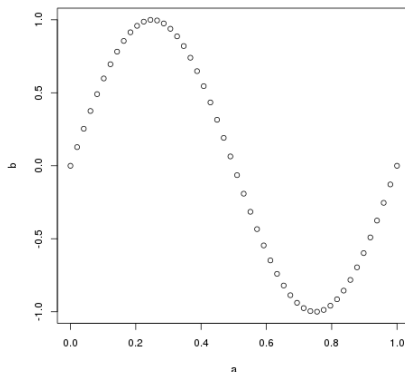
```
> a<-seq(0,1,length=50)
> b<-sin(2*pi*a)
> plot(a,b)
```

La dernière ligne de commande aurait pu s'écrire :

```
> plot(b~a)
```

La fonction `plot` (2/4)

Ces commandes entraînent l'ouverture d'une fenêtre graphique présentant le nuage de points de la fonction $x \mapsto \sin(2\pi x)$:



La fonction `plot` (3/4)

La fonction `plot` est une fonction générique, c'est-à-dire que les résultats qu'elle renvoie diffèrent suivant la classe de l'objet (vecteur, matrice, facteur, ...) sur lequel elle est appelée.

Les fonctions graphiques R comportent de nombreuses options qui permettent de contrôler de façon très fine les graphiques. Pour connaître les arguments de la fonction `plot` utilisés par défaut on entrera :

```
> args(plot.default)
```

Pour lister les autres arguments on entrera :

```
> names(par())
```

Ou bien utiliser :

```
> help(plot)
```

La fonction `plot` (4/4)

Par exemple, l'argument `type` modifie le type de tracé :

`type="l"` comme « line » pour des lignes,

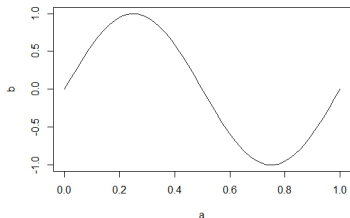
`type="p"` comme « point » (option par défaut) pour des nuages de points,

`type="b"` ou `"o"` pour des points reliés par des lignes,

`type="h"` comme « histogramme »,

`type="s"` et `type="S"` comme « step » pour des escaliers (après ou avant les points).

```
> plot(b~a,type="l")
```



La fonction `curve`

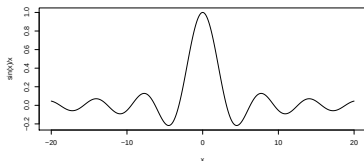
Une façon plus facile pour tracer le graphe d'une fonction est d'utiliser la commande `curve`. La commande `curve(f(x), from, to)` trace la fonction f de $x = \text{from}$ à $x = \text{to}$.

```
> curve(sin(x)/x, -20, 20, n=1000)
```

Cette commande retourne la fonction

$$f(x) = \frac{\sin(x)}{x}$$

entre -20 et 20 sur la base 1000 valeurs de x .



Exportation de graphiques

On peut exporter les graphiques créés, et cela sous plusieurs formats :
jpeg, bmp, pdf, ...

Dans RStudio, on peut exporter le graphique avec le menu « Export »
sous l'onglet « Plots » de la fenêtre **4**.

Quelques exemples (1/9)

Pour illustrer les divers résultats avec la fonction `plot` nous allons utiliser un tableau de données. Pour cela, nous allons importer le fichier de données « ozone.txt » en tapant :

```
> ozone<-read.table("https://r-stat-sc-donnees.github.io/
+ ozone.txt", sep=" ",header=T, dec=".",row.names=1)
```

Le tableau contient des valeurs météorologiques mesurées durant l'été 2001 à Rennes. C'est un data-frame de 13 variables et 112 lignes. Les premières lignes de ce tableau sont :

```
> ozone
      maxO3   T9   T12   T15 Ne9 Ne12 Ne15      Vx9      Vx12      Vx15 maxO3v vent pluie
20010601    87 15.6 18.5 18.4   4   4   8  0.6946 -1.7101 -0.6946    84 Nord  Sec
20010602    82 17.0 18.4 17.7   5   5   7 -4.3301 -4.0000 -3.0000    87 Nord  Sec
20010603    92 15.3 17.6 19.5   2   5   4  2.9544  1.8794  0.5209    82  Est  Sec
...
```

Quelques exemples (2/9)

Nous en allons considérer les variables `maxO3` (maximum d'ozone journalier), `T12` (température à midi), `vent` (direction) et `pluie`.

```
> ozone[c("maxO3", "T12", "vent", "pluie")]  
      max O3    T12    vent    pluie  
20010601      87  18.5   Nord     Sec  
20010602      82  18.4   Nord     Sec  
20010603      92  17.6    Est     Sec  
...
```

On aura le même résultat avec

```
> subset(ozone, select=c("maxO3", "T12", "vent", "pluie"))
```

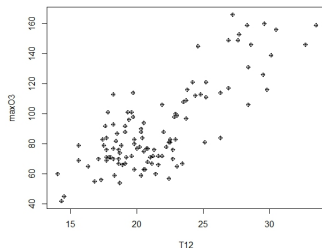
Les variables `vent` (valeurs : Nord, Est, Sud, Ouest) et `pluie` (valeurs : Sec, Pluie) sont qualitatives, les deux autres sont quantitatives.

Rappel : le vecteur qui correspond à la variable `maxO3` est extrait soit

Quelques exemples (3/9)

une variable quantitative en fonction d'une autre variable quantitative

```
> plot(ozone$maxO3~ozone$T12,  
+ pch=10)  
ou > plot(maxO3~T12,data=ozone,  
+ pch=10)
```

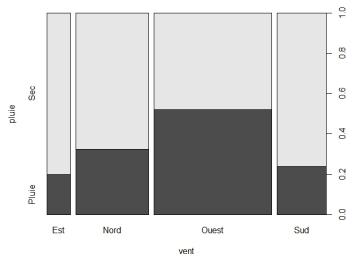


L'argument `pch` permet de préciser la forme des points. Le défaut, `pch=1`, donne des petits cercles, `pch=20` fait des points gras, etc. On peut également choisir un caractère arbitraire, par exemple `pch="?"`.

Quelques exemples (4/9)

une variable qualitative en fonction d'une autre variable qualitative

`> plot(pluie~vent,data=ozone)` Le même graphique aurait pu être obtenu grâce à la fonction `spineplot`.



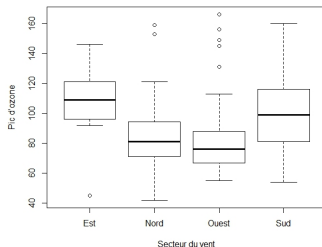
Compte tenu de la nature des variables, la fonction `plot` a choisi d'utiliser `spineplot` car elle est considérée comme plus pertinente.

Quelques exemples (5/9)

une variable quantitative en fonction d'une variable qualitative

```
> plot(maxO3~vent, data=ozone,  
+ xlab="Secteur du vent",  
+ ylab="Pic d'ozone")
```

Le même graphique aurait pu être obtenu grâce à la fonction `boxplot`.



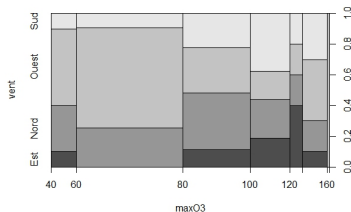
On voit une boîte à moustache (boxplot) avec la médiane, les quantiles de niveau 25% et 75%, le maximum et le minimum des valeurs – après avoir exclu des individus aberrants (outliers).

Quelques exemples (6/9)

une variable qualitative en fonction d'une variable quantitative

```
> plot(vent~maxO3, data=ozone)
```

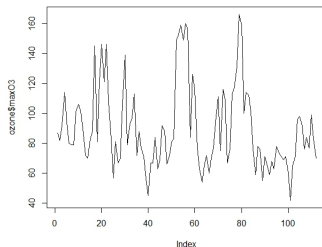
Le même graphique aurait pu être obtenu grâce à la fonction `spineplot`.



Quelques exemples (7/9)

Avec `plot` et ses variantes on peut également représenter une seule variable.

représentation d'une variable quantitative : >
`plot(ozone$maxO3,type="l")`

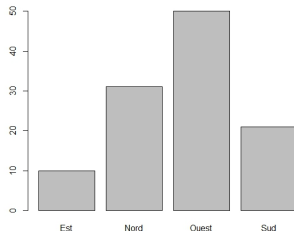


Quelques exemples (8/9)

représentation d'une variable qualitative : >

`plot(ozone$vent)`

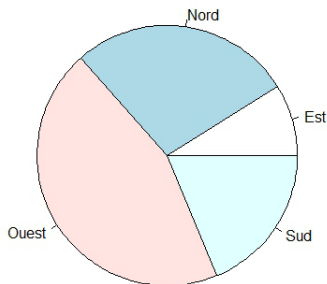
Le même graphique aurait pu être obtenu grâce à la fonction `barplot(table())`. La fonction `table()` transforme le vecteur en un format adapté à la fonction `barplot()`.



Quelques exemples (9/9)

Une autre méthode pour représenter des variables qualitatives est la représentation circulaire appelée également diagramme en camembert que l'on peut obtenir grâce la fonction `pie`.

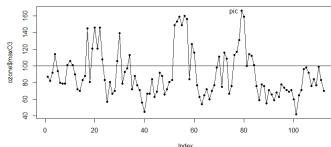
```
> pie(table(ozone$vent))
```



Ajouts aux graphiques

Une fois le graphique tracé, il est possible de le compléter par d'autres informations : ajouts de lignes (`abline` ou `lines`), points (`points`), textes (`text`), flèches (`arrows`), etc. . .

```
> plot(ozone$maxO3,type="l")  
> points(ozone$maxO3,pch=20)  
> abline(h=100)  
> text(which.max(ozone$maxO3),  
+ max(ozone$maxO3),"pic",pos=2)
```



Avec la dernière commande nous avons écrit `pic` là où se trouve la valeur maximale de `maxO3`, et cela à gauche (`pos=2`) du point correspondant.

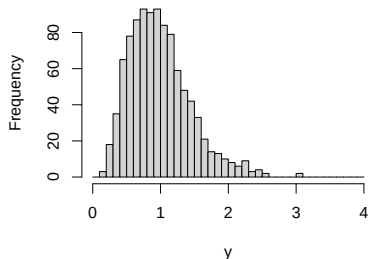
Plusieurs fenêtres

Avec la command `par` on peut ouvrir plusieurs fenêtres en parallèle.

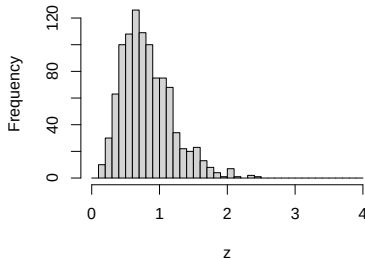
Exemple :

```
> par(mfrow= c(1,2))
```

Histogram of y



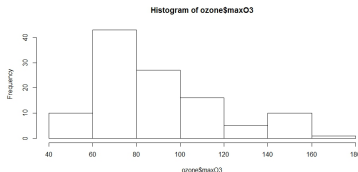
Histogram of z



Histogrammes (1/2)

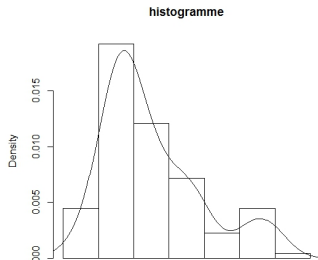
La commande `hist` permet d'obtenir l'histogramme d'une variable aléatoire continue.

```
> hist(ozone$maxO3)
```



Si l'on spécifie l'argument `prob=TRUE` de la fonction `hist`, la commande `lines(density())` permet d'estimer la densité de la variable :

```
> hist(ozone$maxO3,main=
+ "histogramme",prob=T,
+ xlab="Ozone")
> lines(density(ozone$maxO3))
```



Histogrammes (2/2)

Il est possible d'obtenir les résultats numériques qui ont permis de tracer cet histogramme :

```
> (histo<-hist(ozone$maxO3))
```

R nous renvoie alors les différentes informations concernant l'histogramme.

Pour connaître par exemple les bornes des classes de l'histogramme et les effectifs des classes :

```
> histo$breaks  
[1] 40 60 80 100 120 140 160 180
```

```
> histo$counts  
[1] 10 43 27 16 5 10 1
```



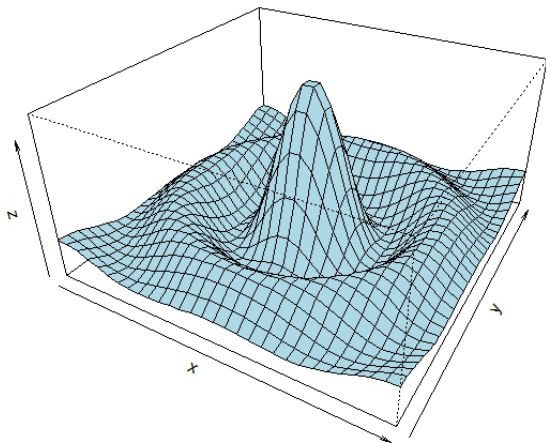
Graphiques 3D (1/2)

Les commandes pour la représentation 3D sur une grille de points sont les fonctions `persp` (3D avec effet de perspective), `contour` (lignes de niveau) et `image` (lignes de niveau avec effet de couleur).

Exemple : La fonction $f(x,y) = 10 \sin(\sqrt{x^2 + y^2}) / \sqrt{x^2 + y^2}$. Nous choisissons une grille régulière carrée comprenant 30 valeurs différentes comprises entre -10 et 10 (sur x et y) :

```
> x <- seq(-10, 10, length=30)
> y <- x
> f <- function(x, y) {r <- sqrt(x^2+y^2); 10 * sin(r)/r}
> z <- outer(x, y, f)
> persp(x, y, z, theta = 30, phi = 30, expand = 0.5,
+ col = "lightblue", main="Chapeau mexicain")
```

Chapeau mexicain



2^{ème} partie

7 Définir une fonction

Définition d'une fonction (1/3)

Plusieurs fonctions sont déjà existantes dans R, mais vous pouvez en créer vous-même si vous en avez besoin.

La valeur d'une fonction peut être le résultat d'un certain nombre de commandes.

On définit une fonction de la manière suivante :

```
fct <- function (arguments) {expressions}
```

où

- *fct* : nom de la fonction,
- *arguments* : liste des arguments séparés par des virgules,
- *expressions* : corps de la fonction, suite d'expressions séparées par des points-virgules.

Définition d'une fonction (2/3)

Si on a une seule expression, les accolades ne sont pas nécessaires.

Exemple 1 : la somme des entiers de m à n :

```
> somme<-function(m,n) sum(m:n)
> somme(3,5)
[1] 12
```

Les expressions peuvent être séparées par un passage à la ligne. Dans ce cas-là, les points-virgules ne sont pas nécessaires.

Exemple 2 : l'écart type d'un échantillon :

```
> ecarttype<-function(x) {
+   y<-sum( (x-mean(x)) ^2)
+   y<-y/ (length(x)-1)
+   y^0.5
+ }
> ecarttype(1:10)
[1] 3.02765
```



Définition d'une fonction (3/3)

La plupart des fonctions sont écrites dans le but de retourner un résultat.

- Par défaut, une fonction retourne le résultat de la dernière expression du corps. Si c'est une affectation, la fonction ne retourne rien.
- On peut également utiliser `return()` dans le corps pour désigner le résultat à retourner.
- Lorsqu'une fonction doit retourner plusieurs résultats, il est en général préférable d'avoir recours à une liste.

Variables locales et globales

Comme la majorité des langages de programmation, R comporte les concepts de variable locale et de variable globale.

- Toute variable définie dans une fonction est locale à cette fonction, c'est-à-dire qu'elle :
 - ▶ n'apparaît pas dans l'espace de travail ;
 - ▶ n'écrase pas une variable du même nom dans l'espace de travail.
- Il est possible de définir une variable dans l'espace de travail depuis une fonction avec l'opérateur d'affectation `<<-`. Il est très rare — et généralement non recommandé — de recourir à de telles variables globales.
- On peut définir une fonction à l'intérieur d'une autre fonction. Cette fonction sera locale à la fonction dans laquelle elle est définie.

Fonctions anonymes

Il n'est pas toujours utile de nommer une fonction. Par exemple, si on l'utilise une seule fois comme argument d'une autre fonction. Dans ce cas, cette fonction joue le rôle de **fonction anonyme**.

Exemple : Le produit externe de deux vecteurs x et y est la matrice dont l'élément (i,j) est $x_i \cdot y_j$. Il peut s'effectuer à l'aide de la fonction `outer`.

```
> outer(1:3, 4:6)
      [,1] [,2] [,3]
[1,]    4    5    6
[2,]    8   10   12
[3,]   12   15   18
```

Si une fonction est donnée comme paramètre optionnel, le résultat est une matrice dont l'élément est $f(x_i, y_j)$.

```
> outer(1:3, 4:6, function(x,y) x*y^2)
      [,1] [,2] [,3]
[1,]   16   25   36
[2,]   32   50   72
```



⑧ Programmer en R

Programmer en R

La programmation en R est basée sur les mêmes principes que d'autres logiciels de calcul scientifique.

En effet, on retrouve les structures classiques de la programmation (boucles `for` ou `while`, condition `if ...else`, etc.)

Mais les structures classiques sont généralement coûteuses en temps de calcul. Certaines alternatives ont été prédéfinies dans R.

La fonction sample

La fonction `sample(x, size, replace=F)` permet de choisir au hasard des éléments (la quantité est `size`) d'un vecteur `x` avec ou sans remise (`replace=T` ou `F`).

Si `x` est un entier, le vecteur `1:x` sera considéré.

```
> sample(1:20, 4)
[1] 10 5 12 19
```

```
> sample(c(1,1,1,2), 16, replace=T)
[1] 2 1 2 1 2 1 1 1 1 1 1 1 1 1 1 1
```

```
> sample(c(1,2), 16, replace=T, prob=c(0.75, 0.25))
[1] 1 2 1 1 1 2 1 1 1 1 1 1 1 1 1 1
```

À chaque évaluation de la fonction, vous obtiendrez des éléments différents (pourvu que vous ne fixiez pas l'amorce avec `set.seed()`).



La fonction replicate

La fonction `replicate` simplifie la syntaxe pour l'exécution répétée d'une expression.

Son usage est particulièrement indiqué pour les simulations. Ainsi, on peut construire une fonction `fun` qui fait tous les calculs d'une simulation, puis obtenir les résultats pour un nombre fixé `n` de simulations avec

```
> replicate(n, fun(...))
```

Exemple : Faire 8 fois l'expérience de choisir 100 réalisations aléatoires suivant une loi normale centrée réduite et en faire la moyenne.

```
> replicate(8, mean(rnorm(100)))
```



La fonction `apply` (1/4)

La fonction `apply` sert à appliquer une fonction quelconque à toutes les marges d'une matrice ou un tableau. La syntaxe de la fonction est la suivante :

```
apply(m, margin, fun, ...),
```

où

- `m` est une matrice ou un tableau ;
- `margin` est un vecteur d'entiers contenant la ou les dimensions de la matrice ou du tableau sur lesquelles la fonction doit s'appliquer ;
- `fun` est la fonction à appliquer.

La fonction `apply` (2/4)

Exemple 1 : Une matrice :

Considérons la matrice `m` constitué de 20 nombres entiers tirés au hasard et sans remise entre 1 et 20 (`sample`).

```
> set.seed(1234)
> (m<-matrix(sample(1:20,20),ncol=4))
      [,1] [,2] [,3] [,4]
[1,]     3    10     7     9
[2,]    12     1     5    17
[3,]    11     4    20    16
[4,]    18     8    15    19
[5,]    14     6     2    13
```

Pour calculer la moyenne de chaque colonne, on utilise :

```
> apply(m,2,mean)
[1] 11.6 5.8 9.8 14.8
```

Ici, le `2` indique que la fonction `mean` porte sur la deuxième dimension de la matrice, à savoir les colonnes.

La fonction `apply` (3/4)

La fonction `apply` sert principalement à calculer des sommaires par ligne (dimension 1) ou par colonne (dimension 2) autres que la somme ou la moyenne (puisque les fonctions `rowSums`, `colSums`, `rowMeans` et `colMeans` existent).

```
> colMeans(m)
[1] 11.6 5.8 9.8 14.8
```

On a bien le même résultat que précédemment.

On peut également définir la fonction `fun` à l'intérieur de la fonction `apply`.

```
> apply(m, 2, function(x) max(x) - min(x))
[1] 15 9 18 10
```

La fonction `apply` (4/4)

Exemple 2 : Un tableau de dimension 3 :

Considérons le tableau `y` constitué de 12 nombres entiers.

```
> (y<-array(seq(1,23,2),c(2,3,2)))
```

Le résultat sera :

	,,1				,,2		
	[, 1]	[, 2]	[, 3]		[, 1]	[, 2]	[, 3]
[1,]	1	5	9	[1,]	13	17	21
[2,]	3	7	11	[2,]	15	19	23

Un exemple de la fonction `apply` sur ce tableau est

```
> apply(y,2:3,sum)
      [,1] [,2]
[1,]     4  28
[2,]    12  36
[3,]    20  44
```

Pour chacune des valeurs possibles de la dimension 2 (les colonnes 1 à 3) et chacune des valeurs possibles de la dimension 3 (les matrices 1 et 2), on applique la fonction `sum` à tous les éléments du tableau.

Les fonctions `lapply` et `sapply` (1/2)

La fonction `lapply`, ou son équivalent `sapply`, applique une même fonction à chaque élément d'un vecteur ou d'une liste.

La fonction

- `lapply` retourne une liste,
- `sapply` retourne un vecteur ou une matrice, si possible.

Leur syntaxe est similaire à celle de `apply` :

```
lapply(x, fun, ...)  
sapply(x, fun, ...)
```

Les fonctions `lapply` et `sapply` (2/2)

Exemple :

```
> (a<-list(b=matrix(1:9,3),c=matrix(1:4,2)))
```

Le résultat sera :

\$b

	[, 1]	[, 2]	[, 3]
[1,]	1	4	7
[2,]	2	5	8
[3,]	3	6	9

\$c

	[, 1]	[, 2]
[1,]	1	3
[2,]	2	4

Les deux fonctions donneront comme résultats si `fun=sum` :

```
> lapply(a, sum)
```

\$b

```
[1] 45
```

\$c

```
[1] 10
```

```
> sapply(a, sum)
```

```
  b    c
```

```
45    10
```



La fonction multidimensionnelle `mapply` (1/2)

Sa syntaxe est

```
mapply(fun, ...)
```

Le résultat est l'application de la fonction `fun` aux premiers éléments de tous les arguments contenus dans « ... », puis à tous les deuxièmes éléments, et ainsi de suite.

Ainsi, si `v` et `w` sont des vecteurs, `mapply(fun, v, w)` retourne sous forme de liste, de vecteur ou de matrice, selon le cas, `fun(v[1], w[1])`, `fun(v[2], w[2])`, etc...

La fonction multidimensionnelle `mapply` (2/2)

Exemple :

```
> mapply(rep, 1:4, 4:1)
[[1]]
[1] 1 1 1 1

[[2]]
[1] 2 2 2

[[3]]
[1] 3 3

[[4]]
[1] 4
```

Les éléments de « ... » sont recyclés au besoin.

Structures classiques (1/3)

Les structures classiques de la programmation existent également en R :

...la boucle `for` :

Exemple : la somme des entiers de 1 à 100 :

```
> x<-0
> for (i in 1:100) {
+ x<-x+i
+ }
> x
[1] 5050
```

Structures classiques (2/3)

...la boucle `while` :

Exemple :

```
> a<-0  
> b<-3  
> while (b>1) {  
+ a<-a+b  
+ b<-b-1  
+ }  
> a  
[1] 5
```

Structures classiques (3/3)

...la condition `if...else` :

Exemple :

```
> age<-21  
> if (age >= 18) {  
+ print("Vous pouvez voter lors d'une élection.")  
+ } else {  
+ print("Vous ne pouvez pas voter lors d'une élection.")  
+ }
```

Il est fortement conseillé d'**éviter** des boucles en R si possible.

Temps d'exécution

Afin de calculer le temps d'exécution d'un ensemble de commandes, on utilise la commande `proc.time()`

```
> time0<- proc.time()  
ensemble de commandes  
> proc.time()-time0
```

⑨ Lois de probabilité usuelles

Lois sous R

Nom de la loi	Nom sous R	Paramètres
Binomiale	<code>binom</code>	<code>size, prob</code>
Binomiale négative	<code>nbinom</code>	<code>size, prob</code>
Cauchy	<code>cauchy</code>	<code>location=0, scale=1</code>
Exponentielle	<code>exp</code>	<code>rate=1</code>
Gamma	<code>gamma</code>	<code>shape, rate=1</code>
Géométrique	<code>geom</code>	<code>prob</code>
Hypergéométrique	<code>hyper</code>	<code>m, n, k</code>
Khi-deux	<code>chisq</code>	<code>df</code>
Normale	<code>norm</code>	<code>mean=0, sd=1</code>
Poisson	<code>pois</code>	<code>lambda</code>
Student	<code>t</code>	<code>df</code>
Uniforme continue	<code>unif</code>	<code>min=0, max=1</code>

Avant d'utiliser une loi il faut bien vérifier sa définition sous R. Par exemple, pour la loi exponentielle, `rate` = λ .



Utilisation des lois (1/4)

Les noms des lois ne sont pas des commandes, elles doivent s'utiliser avec un préfixe.

Pour chacune de ces distributions, vous disposez des quatre préfixes `d`, `p`, `q`, `r` :

- `dnomdist(x, ...)` est la densité $\rho_X(x)$ pour une loi continue et la probabilité $\mathbb{P}(X = x)$ pour une loi discrète ;
- `pnomdist(x, ...)` est la fonction de répartition $F_X(x) = \mathbb{P}(X \leq x)$;
- `qnomdist(y, ...)` est la fonction des quantiles $q_y = F_X^{-1}(y) = \inf\{x \in \mathbb{R} : F_X(x) \geq y\}$;
- `rnomdist(n, ...)` génère n réalisations aléatoires indépendantes de la loi.

Utilisation des lois (2/4)

Exemple 1 : obtenir la probabilité qu'une variable de loi de Poisson de paramètre 1 prenne la valeur 3.

```
> dpois(3,1)
[1] 0.06131324
```

De même pour plusieurs variables :

```
> dpois(c(3, 0, 8), c(1, 4, 10))
[1] 0.06131324 0.01831564 0.11259903
```

Exemple 2 : calculer le quantile d'ordre 0,95 pour la loi normale centrée réduite :

```
> qnorm(.95)
[1] 1.644854
```



Utilisation des lois (3/4)

Exemple 3 : obtenir quatre nombres aléatoires issus de la distribution uniforme entre 0 et 2.

```
> runif(4, max=2)
[1] 0.5305724 1.0703804 0.9986784 1.4701531
```

À chaque évaluation de la fonction, vous obtiendrez des nombres différents !

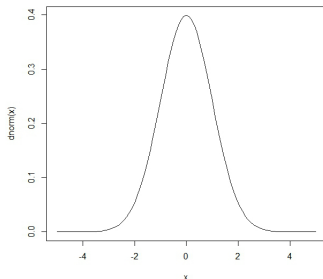
R utilise un générateur de nombres pseudo-aléatoires pour réaliser des simulations et choisit une amorce (ou graine). Il est possible de fixer cette amorce à l'aide de la fonction `set.seed(n)`, `n` doit être un entier. Pour une même valeur de l'amorce `n`, on obtient toujours les mêmes valeurs « aléatoires ».

Utilisation des lois (4/4)

Rappel : Pour représenter des densités, vous pouvez utiliser la fonction `curve`.

```
> curve(dnorm(x),-5,5)
```

Cette commande retourne la densité de la loi normale centrée réduite entre -5 et 5 .



10 Packages

Qu'est-ce qu'un package ?

Un **package** est un ensemble de programmes (fonctions) de R qui complète et augmente les fonctionnalités de R. Par défaut, R charge en mémoire seulement quelques packages de la bibliothèque.

Actuellement, R compte presque 20000 packages (1300 en 2008, 5300 en 2012).

Sous l'onglet « Packages » de RStudio vous verrez tous les packages installés sur l'ordinateur.

Installer un package

D'autres packages peuvent être téléchargés librement sur le réseau du CRAN.

Pour installer un package disponible sur CRAN, il suffit d'utiliser l'onglet « Install » sous l'onglet « Packages » dans la fenêtre **4** de RStudio. L'installation est à faire une seule fois.

Pour utiliser un package dans une session de R il faut le charger par

```
> library(nom.du.package)
```

Certains packages sont en constante évolution. Il est donc intéressant de les mettre à jour de temps en temps. Cela se fait via l'onglet « Update » sous l'onglet « Packages » de RStudio.